Scaling Verification Can Be More Effective than Scaling Policy Learning for Vision-Language-Action Alignment

Jacky Kwok[1] **Xilun Zhang**[1] **Mengdi Xu**[1] **Yuejiang Liu**[1]
Azalia Mirhoseini[1] **Chelsea Finn**[1] **Marco Pavone**[1,2]
[1]Stanford University [2]NVIDIA

https://cover-vla.github.io

Abstract

The long-standing vision of general-purpose robots hinges on their ability to understand and act upon natural language instructions. Vision-Language-Action (VLA) models have made remarkable progress toward this goal, yet their generated actions can still misalign with the given instructions. In this paper, we investigate test-time verification as a means to shrink the "intention-action gap". We first characterize a test-time scaling law for embodied instruction following and demonstrate that jointly scaling the number of paraphrased instructions and generated actions greatly increases test-time sample diversity, often recovering correct actions more efficiently than scaling each dimension independently. To capitalize on these scaling laws, we present CoVer, a contrastive verifier for vision–language–action alignment, and show that our architecture scales gracefully with additional computational resources and data. We then introduce "boot-time compute" and a hierarchical verification inference pipeline for VLAs. At deployment, our framework precomputes a diverse set of paraphrased instructions from a Vision-Language-Model (VLM), repeatedly generates action candidates for each instruction, and then uses a verifier to select the optimal high-level prompt and low-level action chunks. Compared to scaling policy pre-training on the same data, our verification approach yields 22% gains in-distribution and 13% out-of-distribution on the SIMPLER benchmarks, with a further 45% improvements in real-world experiments.

1. Introduction

For robots to be useful in human-centric environments, they must be able to interpret and act upon natural language instructions. Vision-Language-Action (VLA) models, pre-trained on large-scale robotic datasets, have made significant progress towards this goal [4, 19]. However, their widespread deployment is hindered by a critical "intention-action gap": the misalignment between generated actions and the given language instructions. When the policy fails to follow the instruction, this gap can result in costly errors. For instance, a robot tasked with "putting a plastic container into a drawer" might correctly grasp the container but then fail to discriminate between the oven and a nearby drawer, mistakenly placing the container inside the oven. The container could melt or even catch fire. Addressing this fundamental misalignment is essential for deploying robots in real-world settings.

Existing efforts to close this gap have largely focused on scaling policy pre-training, such as augmenting training data with paraphrased instructions [41] or employing larger VLM backbones [2, 12]. However, these approaches typically yield only incremental gains, and performance still degrades severely under simple perturbations [11, 18]. Moreover, scaling policy pre-training often leads to *catastrophic forgetting*, where learning action generation diminishes the VLM's multimodal understanding and reasoning, hindering generalization and semantic understanding [11, 15]. In this paper, we argue that VLA alignment can be more effectively improved through test-time scaling. More specifically, we ask in this work:

Can we enable VLAs to leverage additional computation at test time to improve the alignment between their generated actions and the provided language instructions?

The implications of answering this question extend not only to the generalization capabilities of VLAs, but also to how practitioners should trade off pre-training and test-time compute in robotics. To this end, we first characterize the test-time scaling law for embodied instruction following. Assuming the presence of an oracle verifier, we observe that action error consistently decreases as we scale the number of paraphrased instructions, establishing a clear relationship between linguistic diversity and performance gains. Moreover, we demonstrate that jointly scaling the number of paraphrased instructions and the generated actions constructs a more diverse action proposal distribution. This hybrid sampling approach often recovers correct actions more efficiently than scaling each dimension independently.

To leverage these scaling laws, we seek to develop a robust verifier for both instruction optimization and action verification. Existing verifiers often focus on low-level dynamics [21, 27] and require costly interactions with the environment [24]. To address this, we draw insights from cross-modal alignment [31, 36] and introduce CoVer, a **co**ntrastive approach for **ver**ifying the alignment across vision, language, and action. Our architecture employs two key components: a text-aware visual encoder

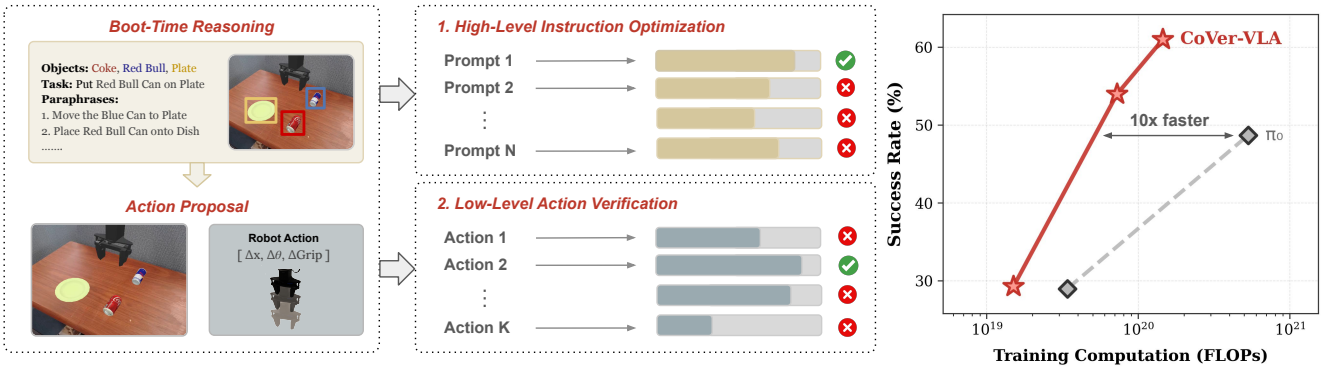

Figure 1. We present **CoVer-VLA**, a contrastive verification approach for vision–language–action alignment. **Left:** Given the initial observation and language instruction, a VLM performs structured reasoning over the scene and precomputes a set of paraphrased instructions at boot time. At each step during deployment, our framework generates a batch of action candidates for each instruction using a VLA. **Middle:** A contrastive verifier then scores all instruction–action pairs and selects the optimal high-level instruction and low-level action chunk **Right:** Compared to prior work on scaling policy learning [3], our approach achieves stronger performance while requiring substantially less compute.

that selectively extracts task-relevant features, and an action encoder that captures long-range temporal dependencies within action chunks. The results show that scaling the number of synthetic instructions, model parameters, negative samples, and verifiers in an ensemble consistently improves verification and downstream retrieval accuracy of CoVer. We train CoVer on 20 million offline samples using a 1B parameter backbone, producing a robust verifier for test-time scaling.

During deployment, our framework first leverages "boot-time compute" to let the robot reason offline. Given the initial observation and language instruction, a VLM performs structured reasoning over the scene—identifying relevant objects, spatial relations, and plausible task decompositions. The resulting reasoning traces are then used to precompute a diverse set of paraphrased instructions, allowing the robot to avoid redundant paraphrase generation during execution. At test time, we employ a hierarchical verification pipeline. This pipeline generates a batch of action candidates for each precomputed instruction with a VLA, scores all instruction-action pairs using CoVer, and then selects the optimal high-level instruction and low-level action chunks for execution. In summary, our contributions are as follows:

1. We characterize the test-time scaling law for embodied instruction following and propose a compute-efficient action sampling method
2. We present a contrastive verifier for vision–language–action alignment and show that our architecture scales gracefully with additional computational resources and data
3. We introduce boot-time compute for offline embodied reasoning and a hierarchical test-time verification pipeline that couples high-level prompt optimization with low-level action chunk selection.
4. We show that pairing VLAs with CoVer yields 22% gains in-distribution and 13% out-of-distribution on the SIMPLER benchmarks, with a further 45% improvement in real-world experiments.

2. Related Work

Vision-Language-Action Models. Recent VLA models, pre-trained on large-scale multimodal data and fine-tuned for visuomotor control, have demonstrated impressive generalization across tasks, objects, and environments [3, 19, 28, 33, 35]. Yet, they still struggle with instruction following: semantically equivalent paraphrases can cause sharp drops in success [11, 18]. Some recent work seeks to mitigate this issue by scaling up model capacity [23], expanding training data [13, 42], and introducing auxiliary objectives to preserve linguistic knowledge [9, 20]. Orthogonal to these training approaches, our work takes a test-time perspective: we treat a user instruction as a distribution over phrasings and verify resulting actions before execution.

Test-Time Scaling. Inference with additional compute has emerged as a promising paradigm for tackling challenging problems across diverse domains, including language reasoning [5, 26, 32, 34], visual understanding [39], and agentic planning [43]. In the context of robot learning, recent studies have demonstrated the effectiveness of optimizing over multiple candidate action sequences to enhance performance [27, 40], consistency [25], and robustness [21]. Such sampling processes can be further accelerated via guidance mechanisms in the latent space [37, 44]. Despite these advances, existing approaches still struggle with instruction following and often incur substantial computational overhead. Our method addresses these challenges through an action verification mechanism explicitly designed for instruction following while enabling acceleration through pre-computation.

Action Verification. Early work on action verification derives signals directly from the policy itself, e.g., prediction uncertainty [14, 41] and temporal consistency [1, 25], yielding lightweight ways to convert prior knowledge into a quality estimator. More recently, a growing body of work has focused on training explicit models for action verification, such as value functions [8, 16] and preference models [21]. Another line of

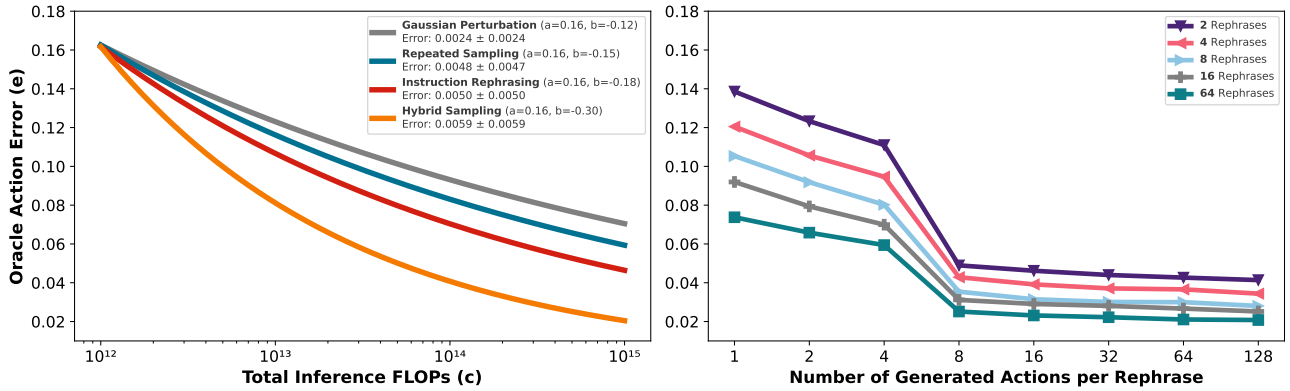

Figure 2. **Test-Time Scaling Law for Embodied Instruction Following.** Compared to prior methods that obtain diverse actions through repeated sampling [27] or Gaussian perturbations [21], we find that instruction rephrasing produces a broader set of action candidates, leading to improved recovery of the correct action. Furthermore, a hybrid test-time scaling strategy that increases both the number of rephrases and the number of sampled actions per rephrase is more effective than either strategy alone. We characterize each sampling approach using a power law, where the logarithm of action error e is a function of the allocated inference compute c: $\log(e) \approx \log(a) + b \cdot \log(c)$.

work decomposes verification into two stages: predicting future states with a dynamics model [30, 40], and then assessing task progress in the predicted states. However, these techniques are still largely centered on low-level dynamics, while high-level instruction following remains a challenge. We instead formulate action verification as a contrastive alignment problem between language and behavior, explicitly targeting instruction-following quality.

3. Test-Time Scaling Analysis

In this section, we characterize the test-time scaling law for embodied instruction following, revealing how linguistic diversity in instructions affects downstream robot policy performance. Following the scheme introduced by Kwok et al. [21], we uniformly sample 1,000 $(s,\ a,\ I)$ tuples from the Bridge V2 dataset [38]. For each tuple, we scale the number of generated action candidates using different sampling strategies and compute the Normalized Root Mean Squared Error (NRMSE) between the ground-truth action a^* and each of the sampled actions $\{a_1, a_2, ..., a_m\}$.

We evaluate four sampling approaches: **Repeated sampling**: actions are repeatedly sampled from a robot policy $\pi(a \mid s,\ I)$ with a positive temperature. **Gaussian perturbation**: a small batch of actions is sampled from the policy $\pi(a \mid s,\ I)$, from which a Gaussian distribution is fit and used to draw all candidate actions. **Instruction rephrasing**: actions are sampled from the policy $\pi(a \mid s, I)$ conditioned on rephrased instructions $\{l_1, l_2, ..., l_k\}$ generated by a VLM. **Hybrid sampling**: instead of generating a single action candidate per rephrased instruction, we fan out and repeatedly sample multiple actions per rephrase. We also find that the relationship between action error and total inference FLOPs follows an exponentiated power law across these sampling methods. For power law fitting, we model the logarithm of action error e as a function of the allocated inference compute.

The results in Figure 2 reveal two key findings: (1) instruction rephrasing consistently yields lower action error compared to vanilla repeated sampling and Gaussian perturbation; and (2) the hybrid approach combining instruction rephrasing with repeated sampling achieves even greater diversity by exploring radically different actions rather than getting stuck in a local minimum.

4. Method

While prior works focus either on policy learning or on atomic-level action verification, our approach introduces a general hierarchical test-time verification and scaling framework that integrates scene reasoning, embodied prompt optimization, and low-level instruction–action verification. Instead of treating the base model's output as final, we jointly select high-level language prompts and low-level action alignment through an optimized latency-aware inference pipeline. In Section 4.1, we formalize our test-time inference framework that jointly optimizes over paraphrased instructions and candidate action chunks. Section 4.2 then details the offline training of CoVer verifier, which provides the alignment signal used during inference. Finally, Section 4.3 presents the complete inference pipeline, including action proposal generation, language-level optimization, and action-level verification.

4.1. Hierarchical Prompt-Action Optimization

We consider a sequential decision-making problem with observation space $\mathcal{O}$, action space $\mathcal{A}$, and natural-language instruction space $\mathcal{L}$. At timestep t, the robot receives an observation $o_t \in \mathcal{O}$ and a user instruction $l \in \mathcal{L}$. A chunk-based VLA policy π produces an action chunk $a_t \sim \pi(a_t \mid o_t, l)$, where a_t may correspond to multiple low-level control steps.

Natural language permits many semantically equivalent paraphrases, yet VLA policies are notoriously sensitive to

phrasing. For a paraphrased instruction l', the induced action $a'_t \sim \pi(a'_t \mid o_t, l')$ may deviate significantly from the intended behavior, revealing a brittleness to linguistic drift. This motivates treating the instruction itself as a decision variable that can be optimized at test time.

Language-level optimization. Rather than committing to a single phrasing, we construct a set with K number of paraphrases:

$$\mathcal{L}_r(l') = \{l'_1, \ldots, l'_K\},$$

all expressing the same user intent. Each l'_k conditions a different action distribution under the fixed base policy. To formalize the objective, we use a conceptual reward function $r(o_t, a, l)$ that measures how well an action a fulfills the semantics of the original instruction l; this reward is *not* computed at test time but serves to define the ideal target behavior. We then aim to select the paraphrase whose induced behavior best aligns with the original intent:

$$l^* = \arg\max_{l' \in \mathcal{L}_r} \mathbb{E}_{a \sim \pi(\cdot \mid o_t, l')} \left[r(o_t, a, l) \right].$$

This reformulates VLA inference as an optimization problem in *language space*, not parameter space.

Action-level optimization. Given a selected paraphrase l^*, sampling a single action from π is unreliable due to bias and noise. We therefore draw M candidate action chunks from the policy, conditioning on the current observation o_t and the selected instruction l^*:

$$a'_j \sim \pi(\cdot \mid o_t, l^*), \qquad j = 1, \ldots, M,$$

We then select the candidate that maximizes semantic alignment with the instruction:

$$a^*_t = \arg\max_{j \in [M]} \mathcal{V}_\theta \left(o_t, h_t, l^*, a'_j \right),$$

where $\mathcal{V}_\theta$ estimates vision–language–action alignment, and $h_t \in \mathcal{A}^W$ denotes the recent action history (e.g., the previous chunk or last W action steps), providing temporal context to the verifier.

This view unifies language refinement and action verification: the system first searches for the paraphrase whose induced action distribution aligns with the user intent, then verifies individual action candidates within that distribution. Treating both language and action as inference-time decision variables enables the robot to correct prompt misalignment, counteract policy brittleness, and improve robustness without modifying the underlying VLA policy.

4.2. Offline Verifier Training

The objective of verifier $\mathcal{V}_\theta$ is to assess the semantic alignment between visual observations, instruction language, and action sequences. A central challenge in training a VLA verifier is that robotic datasets contain only successful demonstrations,

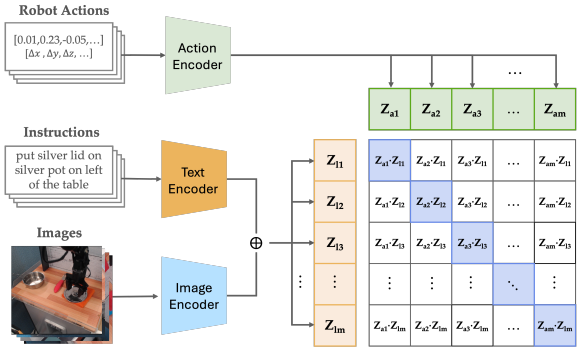

Figure 3. **Overview of CoVer Training Strategy.** CoVer learns a joint embedding space aligning visual observations, language instructions, and robot actions through contrastive pre-training. Image and text encoders extract task-relevant visual–linguistic features, which are fused into text-aware visual representations. An action encoder maps action sequences into the same embedding space, enabling cross-modal alignment between intended instruction and executed behaviors.

Algorithm 1 Verifier Training with Rephrase Augmentation

Require: Offline trajectories $\mathcal{D} = \{(o_t, h_t, l, a_t)\}_{t=1}^{T}$; batch size B; Augmented Instruction Set $\mathcal{I}$

Initialize augmented dataset $\mathcal{D}_{\text{aug}} \leftarrow \emptyset$

Stage 1: Paraphrase Augmentation

for $(o_t, h_t, l, a_t) \in \mathcal{D}$ **do**

  for l_n^{oxe} in $\mathcal{I}(l)$ **do**

    $\mathcal{D}_{\text{aug}} \leftarrow \mathcal{D}_{\text{aug}} \cup \{(o_t, h_t, l_n^{oxe}, a_t)\}$

Stage 2: Verifier Training

Initialize parameters θ

while not converged **do**

  Sample minibatch $\{(o_i, h_i, l_i, a_i)\}_{i=1}^{B} \sim \mathcal{D}_{\text{aug}}$

  for $i = 1..B$ **do**

    $\mathbf{F}_i = \mathbf{F}_{\text{combined}}(o_i, l_i)$

    $\mathbf{A}_i = \mathbf{A}(h_i, a_i)$

    Normalize: $\hat{\mathbf{f}}_i = \mathbf{F}_i / \|\mathbf{F}_i\|_2, \quad \hat{\mathbf{a}}_i = \mathbf{A}_i / \|\mathbf{A}_i\|_2$

    Compute pairwise similarities $s_{i,j} = \langle \hat{\mathbf{f}}_i, \hat{\mathbf{a}}_j \rangle$

    $\mathcal{L}_i^{f \to a} = -\log \frac{\exp(s_{i,i})}{\sum_{j=1}^{B} \exp(s_{i,j})}$

    $\mathcal{L}_i^{a \to f} = -\log \frac{\exp(s_{i,i})}{\sum_{j=1}^{B} \exp(s_{j,i})}$

  $\mathcal{L}_{\text{InfoNCE}} = \frac{1}{2B} \sum_{i=1}^{B} (\mathcal{L}_i^{f \to a} + \mathcal{L}_i^{a \to f})$

  $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{InfoNCE}}$

providing no direct supervision indicating when an action is semantically *misaligned* with an instruction. Constructing negative examples is non-trivial [41]: synthesizing incorrect actions often produces unrealistic motions, while manually annotating failures is prohibitively expensive. Contrastive learning [31, 36] offers a natural solution by treating other actions in the batch as implicit negatives, allowing the model to learn alignment structure without curated failure labels. Our training pipeline consists of two stages: (i) augmenting the instruction space with diverse paraphrases and (ii) contrastive learning on the augmented dataset. The detailed algorithm is shown in Algorithm 1.

Paraphrase Augmentation. To address the linguistic sensitivity of VLA policies, we expand each original instruction solely in language space, leaving observations and actions fixed. The training language augmentation is obtained from Open-X Embodiment [7] datasets $\mathcal{I}$, where each original task instruction set $\mathcal{I}(l)$ corresponds to N rephrases. Each selected paraphrase l_n^{oxe} from $\mathcal{I}(l)$ is then paired with the same observation o_t, and ground-truth action sequence that consists of short-term action history h_t and future action chunk a_t to form additional training tuples. Paraphrases enable the verifier to encounter multiple linguistic realizations of the same underlying intent. This procedure enlarges the effective language coverage of the dataset without altering the action distribution, and equips the verifier with the ability to distinguish true semantic equivalence from phrasing-induced discrepancies that often mislead the base VLA policy. Though the same paraphrase augmentation technique has been developed for policy learning [10, 11], we demonstrate that using the same data budget to train a verifier would be more effective than directly augmenting the policy training dataset.

Verifier Training and Architecture. The verifier aims to estimate the alignment between visual–textual and action representations. Visual inputs and language tokens are encoded with pre-trained SigLIP2 encoders [36], then fused via text-aware visual attention to obtain instruction-relevant features. Vision and text encoders are frozen during verifier training to preserve the web-scale knowledge [17]. The resulting fused representation $\mathbf{F}_{\text{combined}}$ captures visual–language context. The action sequence, which contains short-term history and future chunks, is processed by a transformer encoder to better capture the temporal features of low-level behaviors [25]. The fused vision–language representation $\mathbf{F}_{\text{combined}}$ and the action embedding $\mathbf{A}$ are then ℓ_2-normalized to get $\hat{\mathbf{f}}$ and $\hat{\mathbf{a}}$ respectively. Their similarity defines the alignment score: $s(\hat{\mathbf{f}},\hat{\mathbf{a}}) = \langle \hat{\mathbf{f}},\hat{\mathbf{a}} \rangle$. Given a minibatch of B tuples $\{(o_i,h_i,l_i,a_i)\}_{i=1}^{B}$, the verifier is trained with InfoNCE [29] objective:

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{2B}\sum_{i=1}^{B}\left[\log\frac{\exp\big(s(\hat{\mathbf{f}}_i,\hat{\mathbf{a}}_i)\big)}{\sum_{j=1}^{B}\exp\big(s(\hat{\mathbf{f}}_i,\hat{\mathbf{a}}_j)\big)} + \log\frac{\exp\big(s(\hat{\mathbf{a}}_i,\hat{\mathbf{f}}_i)\big)}{\sum_{j=1}^{B}\exp\big(s(\hat{\mathbf{a}}_i,\hat{\mathbf{f}}_j)\big)}\right]. \tag{1}$$

This symmetrical formulation aligns vision–language embeddings $\hat{\mathbf{f}}$ with action embeddings $\hat{\mathbf{a}}$ in both directions. By treating all other pairs in the batch as *implicit negatives*, it leverages the diversity of each minibatch to learn robust fine-grained correspondences without requiring explicit failure labels or hand-crafted counterexamples. Such in-batch contrastive structure enables the verifier to discover meaningful distinctions between semantically aligned and misaligned behaviors, leading to more stable and cycle-consistent vision–language–action grounding during test-time verification. The verifier structure is shown in Figure 3.

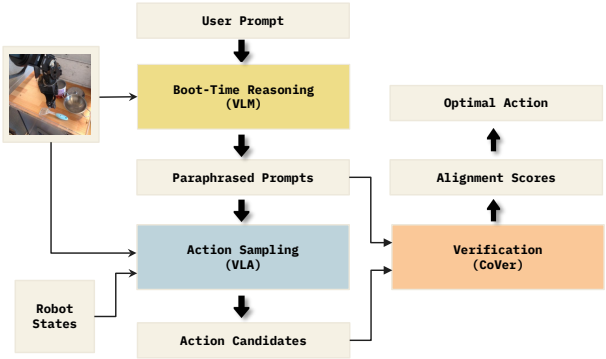

Figure 4. **Hierarchical Test-time Verification Pipeline.** At deployment, the system performs hierarchical optimization over language and action spaces. Given a user prompt and initial observation, a VLM generates paraphrased prompts and scene reasoning at *boot time*. The base VLA policy samples candidate action chunks conditioned on each paraphrase, while CoVer evaluates their semantic alignment with the instruction and image observation. Alignment scores guide the selection of both the most reliable paraphrase and the optimal action chunk.

4.3. Test-time Verification

In Section 4.2, we explored the advantages of contrastive training for vision-language-action alignment, which enables zero-shot verification for both instruction and actions. Such bidirectional feature benefits the verification process to be more flexible and comprehensive. CoVer-VLA provides test-time verification that is robust to language-induced action drift, while adding only minimal latency from proposal generation and verification. CoVer-VLA casts inference as a hierarchical verification problem as shown in Figure 4. The system first evaluates on the language level, selecting the instruction whose induced action distribution is most semantically reliable, and then selects the best action chunk conditioned on that instruction. This hierarchical structure enables the robot to update its active language prompt online and to filter action proposals using a learned alignment score, improving robustness without altering the underlying VLA policy. To support this procedure, we first introduce boot-time paraphrase generation and caching, that significantly boost the runtime efficiency by bring scene reasoning offline. Follow with the details on batched action proposals that enable efficient search over both languages and actions. The resulting pipeline preserves robustness without compromising real-time control, and the full procedure is summarized in Algorithm 2.

Boot-time paraphrase generation and caching. To efficiently handle linguistic variability, we expand each free-form instruction l' into K paraphrases using an off-the-shelf VLM. The VLM takes the initial scene image o_0 and user instruction l' as input. It generates both scene-level reasoning and paraphrased command variants $\{l'_k\}_{k=1}^{K}$. Leveraging the VLM's reasoning capabilities incorporates web-scale knowledge into the paraphrase generation process. Running the VLM on-the-fly, however, is computationally expensive and can introduce undesirable latency or motion discontinuities during robot control. Because

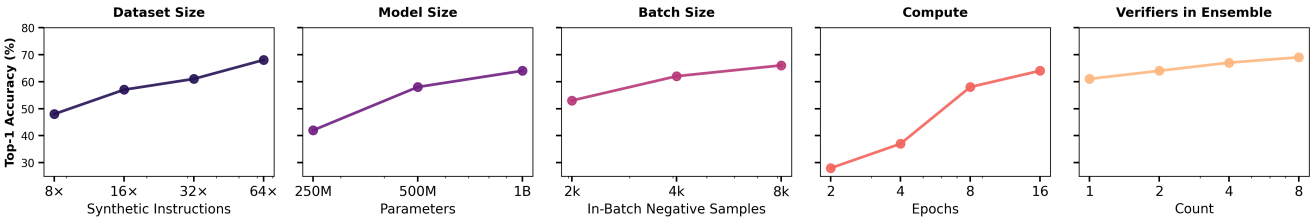

Figure 5. **Verifier Scaling Results.** We demonstrate that our architecture scales gracefully with additional computational resources and data. The top-1 action-retrieval accuracy consistently improves as we scale the number of synthetic instructions, model parameters, negative samples, training compute, and the number of verifiers in the ensemble.

Algorithm 2 Hierarchical Test-time Verification

Input: base policy π, verifier ensemble $\mathcal{V}_\theta$, user instruction l', num of rephrases K, num of action samples M
Boot-time: generate paraphrases $\{l'_k\}_{k=1}^{K} \leftarrow \text{VLM}(o_0, l')$; cache embeddings
while episode not finished **do**
  1. Sample proposal actions
  for $k=1$ to K **do**
    for $j=1$ to M **do**
      $a'_{k,j} \sim \pi(\cdot \mid o_t, l'_k)$
  2. Score proposals
  $s_{k,j} = \mathcal{V}_\theta(o_t, h_t, l'_k, a'_{k,j})$
  3. Select paraphrase (language-level)
  $S_k = \frac{1}{M}\sum_{j=1}^{M} s_{k,j} \qquad k^* = \text{argmax}_k S_k$
  4. Select action (action-level)
  $j^* = \text{argmax}_j s_{k^*,j}$
  Execute a'_{k^*,j^*} and update $(o_{t+\Delta}, h_{t+\Delta})$

user intent is typically stable throughout an episode, generating new paraphrases mid-rollout offers limited benefits. Instead, we perform paraphrase generation and embedding computation entirely at boot time. By caching paraphrases embeddings before execution, we shift the heaviest computations off the critical path and ensure that retrieving paraphrase features at inference time incurs negligible overhead. This allows the controller to evaluate paraphrastic variants efficiently at test time, enabling robust test-time optimization without compromising control smoothness.

Inference with batched action proposals. With paraphrases cached and a verifier in place, we perform chunk-level optimization by jointly searching over paraphrased instructions and candidate action chunks. Let $\{l'_k\}_{k=1}^{K}$ denote the K paraphrases generated at boot time, with $l'_1 = l'$. At each chunk boundary, the base VLA policy induces a distribution over action chunks, $a \sim \pi(\cdot \mid o_t, l'_k)$, from which we sample M candidates for each paraphrase. This yields $K \times M$ proposals:

$$a'_{k,j} \sim \pi(\cdot \mid o_t, l'_k), \qquad k=1,\ldots,K,\ j=1,\ldots,M.$$

Each proposal is then evaluated by the verifier ensemble,

$$s_{k,j} = \mathcal{V}_\theta(o_t, h_t, l'_k, a'_{k,j}),$$

producing a semantic alignment score for every (paraphrase, action) pair. To determine which paraphrase induces the most reliable action distribution, we take the average scores across all M actions from the same language :

$$S_k = \frac{1}{M}\sum_{j=1}^{M} s_{k,j}, \qquad k^* = \underset{k}{\text{argmax}}\, S_k.$$

The chosen paraphrase l'_{k^*} becomes the active language for this chunk. Within the selected paraphrase, the controller chooses the highest-scoring action candidate:

$$j^* = \underset{j}{\text{argmax}}\, s_{k^*,j}.$$

The selected action chunk a'_{k^*,j^*} is executed, and the state $(o_{t+\Delta}, h_{t+\Delta})$ is updated accordingly. This procedure repeats at each chunk boundary, forming a closed-loop optimization that continually adapts both the instruction and the executed action.

5. Experiments

5.1. Verifier Scaling Results

In this section, we investigate the scaling behavior of the CoVer verifier. We conduct thorough studies to explore the impact of five key dimensions: model size, dataset size, batch size, training compute, and ensemble size. Detailed specifications regarding architecture and compute usage are provided in the Appendix.

We first evaluate how scaling synthetic instructions and model parameters affects verifier performance. As shown in Figure 4, CoVer exhibits consistent scaling trends: every increase in dataset size (from $8\times$ to $64\times$) or in model capacity (from 250M to 1B parameters) leads to steady improvements in top-1 retrieval accuracy. This provides strong empirical evidence that our contrastive approach effectively capitalizes on scaling.

We also investigate the effects of scaling batch size and training epochs. Because our verifier relies on contrastive learning, the number of in-batch negative samples is critical for learning robust decision boundaries. We find that larger batch sizes (scaling from 2,048 to 8,192) provide a richer set of negative examples, thereby facilitating better convergence. Similarly, extending training epochs exposes the model to more diverse negative samples, leading to improved results.

Finally, we explore test-time ensembling as a scaling dimension. Specifically, we train multiple verifiers with

identical architectures and data budgets, differing only in their random seeds. During inference, we average the image, text, and action embeddings across these verifiers before computing the cosine similarity between modalities. We find that action retrieval accuracy consistently improves as the ensemble size increases (from 1 to 8). These gains stem from variance reduction, as the ensemble averages out individual model biases.

5.2. Implementation Details

Our final CoVer verifier is a 1B-parameter model trained with a batch size of 32,768 on the augmented Bridge V2 dataset [38] containing 16× synthetic instructions. Training was conducted for a total of 2k steps using 8 NVIDIA H200 GPUs. For deployment, we utilize an ensemble of 3 verifiers to balance robustness and computational overhead.

5.3. Evaluation Setup

We evaluate CoVer-VLA across both simulated and real-world settings, focusing on robustness to linguistic variation and generalization on out-of-distribution environments. Our primary benchmark is the SIMPLER benchmark [22], which includes four in-distribution (ID) manipulation tasks and three OOD variants containing distractor objects and clutter [10]. For language robustness, we report performance under challenging red-teaming instructions generated by ERT [18]. To evaluate the standard in-distribution setting, we run the standard SIMPLER benchmark [22] on four widowX tasks. With the OOD environments, we adopt the environments developed from Interleave-VLA [10], "Redbull on Plate", "Zucchini on Towel", and "Tennis in basket". The OOD environments contains multiple objects in the scene, where the VLA cannot rely solely on visual inputs and must also reason over the object information in the instructions. To evaluate the model performance, we take the most popular chunk-based VLA model π_0 as our base VLA models. For real-world experiments, we evaluated on two tasks "pepto bismol on plate" and "redbull on plate", to further demonstrate the real-world practical deployment.

Baselines and Ablations. We compare CoVer-VLA against five variants built on the same π_0 backbone to disentangle the effects of training-time augmentation and test-time verification. (1) π_0 denotes the standard chunk-based VLA policy fine-tuned on BridgeV2 without instruction augmentation or verification. (2) π_0+Inst. Aug. [11] represents π_0 finetuned on instruction-augmented datasets. (3) RoboMonkey [21] applies a 7B-scale verifier with action resampling for test-time scaling, serving as the strongest prior method without hierarchical reasoning. (4) CoVer-VLA introduces our verifier-based inference that jointly optimizes over paraphrases and action chunks at test time. (5) π_0+Rand. Reph. uses a single random paraphrase without verification to isolate the role of language selection. (6) CoVer-VLA+Inst. Aug. combines both training-time augmentation and our hierarchical test-time verifier to examine their complementarity. Together, these baselines allow us to disentangle the contributions of (i) training-time instruction augmentation, (ii) test-time verification of instructions and actions, and (iii) verifier-guided hierarchical optimization. This enables a comprehensive assessment of CoVer 's robustness and generalization benefits.

5.4. Simulation Evaluation Results

Figure 6 summarizes performance across four ID tasks and three OOD tasks under red-teaming instructions. Due to training distribution shift, Robomonkey fails to select optimal actions given challenging instructions. For all the other ablations, we observe different levels of performance gain compared with base π_0 model. We highlight four key findings.

(1) Training-time augmentation alone provides modest performance gain. Instruction augmentation provides minimal gains over the base policy π_0 (ID: 41.5→44.0). However, it offers modest benefit on OOD tasks compared with base π_0 policy. Such results suggest that augmenting policy learning can indeed improve the task policy while countering challenging rephrases.

(2) Random paraphrases validate VLM rephrases but lack control. Using a single random VLM-generated paraphrase yields slightly better ID performance than π_0 (42.3 vs. 41.5), confirming that VLM rephrasings can indeed help the policy. However, its OOD performance drops (28.7 vs. 29.7), and variance across tasks is extreme (e.g., 78% on *Eggplant in Basket* but only 1% on *Block Stacking*). This shows that paraphrase choice *strongly* affects the resulting action distribution: some rephrasings help, while others catastrophically mislead the policy. Random selection therefore lacks the control and reliability that CoVer 's language-level scoring provides.

(3) CoVer-VLA delivers substantial gains and complements augmented policy learning. Applying CoVer on top of π_0 significantly boosts robustness (15% and 31% performance gain on ID and OOD environment respectively), outperforming π_0 with instruction augmentation by significant margin (15% on ID and 12% on OOD). By jointly selecting the most semantically aligned paraphrase and verifying action chunks, CoVer-VLA recovers correct behavior even under heavily perturbed instructions. Combining Inst. Aug. with CoVer achieves the strongest results overall (65.5% ID, 62.0% OOD), indicating that training-time linguistic diversity and test-time hierarchical verification are complementary.

(4) CoVer introduces moderate latency overhead with a favorable accuracy–latency trade-off. The latency analysis is shown in Table 1. the base π_0 policy runs at 344ms per step, while CoVer-VLA, with batch size of 40, records 853ms latency on VLA, and 7.5ms from the action encoder. Compared to the base policy, CoVer-VLA has about **2.5× inference overhead**, which brings **65% relative performance gain** on average. This modest latency increase remains well within real-time constraints on robotic system deployment, confirming that CoVer's verifier computation can be efficiently parallelized to achieve substantially higher robustness without prohibitive runtime cost.

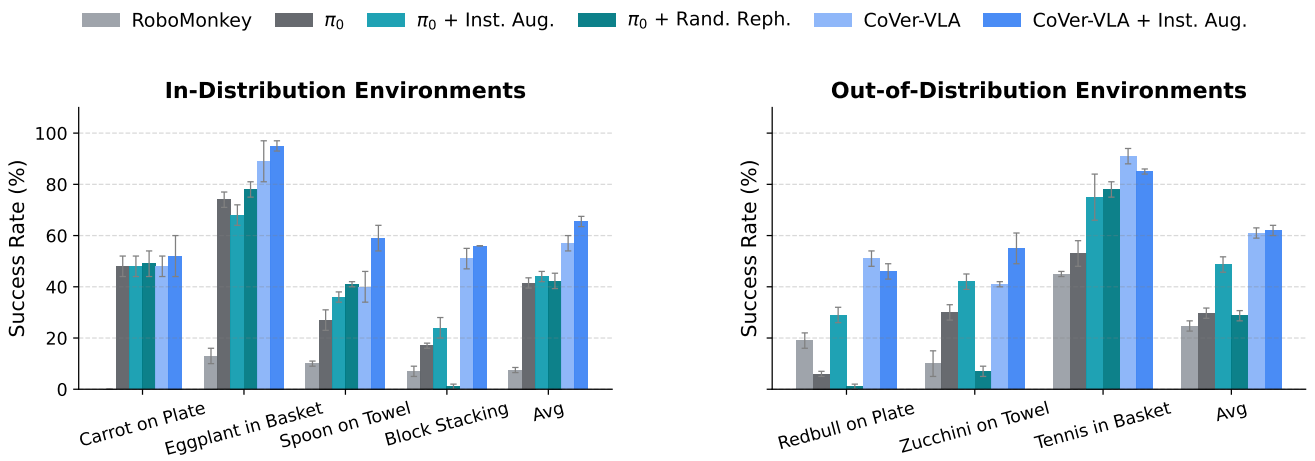

Figure 6. **SIMPLER benchmark evaluation.** Mean success rate ($\pm$ standard deviation) across 50 rollouts and 3 seeds on four ID and three OOD manipulation tasks. π_0 denotes the base chunk-based VLA policy; variants include training-time instruction augmentation (Inst. Aug.) or random paraphrasing (Rand. Reph.). **RoboMonkey** [21] serves as a test-time verification baseline using a large-scale VLM verifier and resampling strategy. **CoVer-VLA** and **CoVer-VLA + Inst. Aug.** achieve the highest scores across both ID and OOD conditions, highlighting the benefits of verifier-guided hierarchical test-time optimization for robust vision-language-action verification. The results are evaluated based on 8 rephrases and 5 sampled actions per instruction.

Batch Size	π_0 (ms)	CoVer (ms)	π_0 + CoVer (ms)
1	344	7	344
2	346	7	353
4	374	8	382
8	411	8	419
16	523	8	531
32	748	8	756

Table 1. Latency (milliseconds) across batch sizes running on RTX-4090 GPU. Since the image-text encoder can run in parallel with the forward pass of π_0, the end-to-end latency of our pipeline only consists of batched inference with π_0 and the action encoder of CoVer.

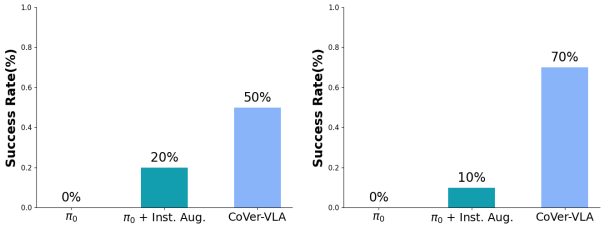

Figure 7. **Real-World Evaluate Results.** Success rates (top) show CoVer-VLA outperforming baseline policies by significant margin. The bottom row illustrates the real-world experiment setup and language instructions.

Overall, these results demonstrate that viewing language as a test-time decision variable and jointly optimizing it with action proposals via CoVer-VLA improves robustness to linguistic variation and strengthens generalization in OOD environments, thereby further advancing the real-world deployment of VLA models.

5.5. Real-World Evaluation Results

We further evaluate CoVer-VLA in two real-world manipulation tasks as shown in Figure 7. CoVer-VLA substantially outperforms the policy-learning baseline, by 30% and 60% success respectively. CoVer-VLA consistently show the correct intention to accomplish the task, where the other baselines always fail to identify the correct object. We observed that the base π_0 model often failed to initiate motion under challenging scenes and instructions, resulting in 0% success. These gains highlight that test-time verification is crucial in real-world settings, where minor linguistic or visual variations often derail standard policies.

6. Conclusion

In this paper, we presented CoVer, a novel contrastive-based verifier and hierarchical test-time scaling framework that bridges the "intention–action gap" for VLAs. CoVer achieves substantial performance improvements across both simulated and real-world settings, particularly under out-of-distribution conditions. Our findings demonstrate that allocating compute to reasoning and verification at deployment can be more effective than scaling policy training alone, providing a promising direction for robust policy deployment in the real world. While our study focuses on applying the verifier for test-time scaling, the same principle can extend beyond inference optimizations such as data curation [6]. Additionally, since our framework samples multiple actions from the base policy, it introduces additional computational overhead. Although parallel execution and batch inference mitigate this cost, CoVer may still be less suitable for tasks requiring high-frequency control. Future work could explore more efficient architectures for the base policy and verifier to enable broader use of test-time scaling in real-world robotic settings.

References

[1] Christopher Agia, Rohan Sinha, Jingyun Yang, Zi-ang Cao, Rika Antonova, Marco Pavone, and Jeannette Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *8th Annual Conference on Robot Learning*, 2024. 2

[2] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, et al. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*, 2024. 1

[3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164*, 2024. 2

[4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023. 1

[5] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. *arXiv preprint arXiv:2407.21787*, 2024. 2

[6] Annie S Chen, Alec M Lessing, Yuejiang Liu, and Chelsea Finn. Curating demonstrations using online experience. *arXiv preprint arXiv:2503.03707*, 2025. 8

[7] Open X.-Embodiment Collaboration, Abhishek Padalkar, Acorn Pooley, Ajay Mandlekar, Ajinkya Jain, Albert Tung, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anikait Singh, Animesh Garg, Anthony Brohan, Antonin Raffin, Ayzaan Wahid, Ben Burgess-Limerick, Beomjoon Kim, Bernhard Schölkopf, Brian Ichter, Cewu Lu, Charles Xu, Chelsea Finn, Chenfeng Xu, Cheng Chi, Chenguang Huang, Christine Chan, Chuer Pan, Chuyuan Fu, Coline Devin, Danny Driess, Deepak Pathak, Dhruv Shah, Dieter Büchler, Dmitry Kalashnikov, Dorsa Sadigh, Edward Johns, Federico Ceola, Fei Xia, Freek Stulp, Gaoyue Zhou, Gaurav S. Sukhatme, Gautam Salhotra, Ge Yan, Giulio Schiavi, Gregory Kahn, Hao Su, Hao-Shu Fang, Haochen Shi, Heni Ben Amor, Henrik I. Christensen, Hiroki Furuta, Homer Walke, Hongjie Fang, Igor Mordatch, Ilija Radosavovic, Isabel Leal, Jacky Liang, Jad Abou-Chakra, Jaehyung Kim, Jan Peters, Jan Schneider, Jasmine Hsu, Jeannette Bohg, Jeffrey Bingham, Jiajun Wu, Jialin Wu, Jianlan Luo, Jiayuan Gu, Jie Tan, Jihoon Oh, Jitendra Malik, Jonathan Booher, Jonathan Tompson, Jonathan Yang, Joseph J. Lim, João Silvério, Junhyek Han, Kanishka Rao, Karl Pertsch, Karol Hausman, Keegan Go, Keerthana Gopalakrishnan, Ken Goldberg, Kendra Byrne, Kenneth Oslund, Kento Kawaharazuka, Kevin Zhang, Krishan Rana, Krishnan Srinivasan, Lawrence Yunliang Chen, Lerrel Pinto, Li Fei-Fei, Liam Tan, Lionel Ott, Lisa Lee, Masayoshi Tomizuka, Max Spero, Maximilian Du, Michael Ahn, Mingtong Zhang, Mingyu Ding, Mohan Kumar Srirama, Mohit Sharma, Moo Jin Kim, Naoaki Kanazawa, Nicklas Hansen, Nicolas Heess, Nikhil J. Joshi, Niko Suenderhauf, Norman Di Palo, Nur Muhammad Mahi Shafiullah, Oier Mees, Oliver Kroemer, Pannag R. Sanketi, Paul Wohlhart, Peng Xu, Pierre Sermanet, Priya Sundaresan, Quan Vuong, Rafael Rafailov, Ran Tian, Ria Doshi, Roberto Martín-Martín, Russell Mendonca, Rutav Shah, Ryan Hoque, Ryan Julian, Samuel Bustamante, Sean Kirmani, Sergey Levine, Sherry Moore, Shikhar Bahl, Shivin Dass, Shubham Sonawani, Shuran Song, Sichun Xu, Siddhant Haldar, Simeon Adebola, Simon Guist, Soroush Nasiriany, Stefan Schaal, Stefan Welker, Stephen Tian, Sudeep Dasari, Suneel Belkhale, Takayuki Osa, Tatsuya Harada, Tatsuya Matsushima, Ted Xiao, Tianhe Yu, Tianli Ding, Todor Davchev, Tony Z. Zhao, Travis Armstrong, Trevor Darrell, Vidhi Jain, Vincent Vanhoucke, Wei Zhan, Wenxuan Zhou, Wolfram Burgard, Xi Chen, Xiaolong Wang, Xinghao Zhu, Xuanlin Li, Yao Lu, Yevgen Chebotar, Yifan Zhou, Yifeng Zhu, Ying Xu, Yixuan Wang, Yonatan Bisk, Yoonyoung Cho, Youngwoon Lee, Yuchen Cui, Yueh-Hua Wu, Yujin Tang, Yuke Zhu, Yunzhu Li, Yusuke Iwasawa, Yutaka Matsuo, Zhuo Xu, and Zichen Jeff Cui. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. *arXiv preprint arXiv:2310.08864*, 2023. 5

[8] Perry Dong, Suvir Mirchandani, Dorsa Sadigh, and Chelsea Finn. What Matters for Batch Online Reinforcement Learning in Robotics? *arXiv preprint*, 2025. 2

[9] Danny Driess, Jost Tobias Springenberg, Brian Ichter, Lili Yu, Adrian Li-Bell, Karl Pertsch, Allen Z. Ren, Homer Walke, Quan Vuong, Lucy Xiaoyang Shi, and Sergey Levine. Knowledge Insulating Vision-Language-Action Models: Train Fast, Run Fast, Generalize Better. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. 2

[10] Cunxin Fan, Xiaosong Jia, Yihang Sun, Yixiao Wang, Jianglan Wei, Ziyang Gong, Xiangyu Zhao, Masayoshi Tomizuka, Xue Yang, Junchi Yan, et al. Interleave-vla: Enhancing robot manipulation with interleaved image-text instructions. *arXiv preprint arXiv:2505.02152*, 2025. 5, 7

[11] Irving Fang, Juexiao Zhang, Shengbang Tong, and Chen Feng. From intention to execution: Probing the generalization boundaries of vision-language-action models. *arXiv preprint arXiv:2506.09930*, 2025. 1, 2, 5, 7

[12] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. 1

[13] Shresth Grover, Akshay Gopalkrishnan, Bo Ai, Henrik I. Christensen, Hao Su, and Xuanlin Li. Enhancing Generalization in Vision-Language-Action Models by Preserving Pretrained Representations. *arXiv preprint arXiv:2509.11417*, 2025. 2

[14] Qiao Gu, Yuanliang Ju, Shengxiang Sun, Igor Gilitschenski, Haruki Nishimura, Masha Itkina, and Florian Shkurti. Safe:

Multitask failure detection for vision-language-action models. *arXiv preprint arXiv:2506.09937*, 2025. 2

[15] Asher J. Hancock, Xindi Wu, Lihan Zha, Olga Russakovsky, and Anirudha Majumdar. Actions as language: Fine-tuning vlms into vlas without catastrophic forgetting, 2025. 1

[16] Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jakub Grudzien Kuba, and Sergey Levine. IDQL: Implicit Q-Learning as an Actor-Critic Method with Diffusion Policies. *arXiv preprint arXiv:2304.10573*, 2023. 2

[17] Huang Huang, Fangchen Liu, Letian Fu, Tingfan Wu, Mustafa Mukadam, Jitendra Malik, Ken Goldberg, and Pieter Abbeel. Otter: A vision-language-action model with text-aware visual feature extraction. *arXiv preprint arXiv:2503.03734*, 2025. 5

[18] Sathwik Karnik, Zhang-Wei Hong, Nishant Abhangi, Yen-Chen Lin, Tsun-Hsuan Wang, Christophe Dupuy, Rahul Gupta, and Pulkit Agrawal. Embodied red teaming for auditing robotic foundation models, 2025. 1, 2, 7

[19] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model, 2024. 1, 2

[20] Taeyoung Kim, Jimin Lee, Myungkyu Koo, Dongyoung Kim, Kyungmin Lee, Changyeon Kim, Younggyo Seo, and Jinwoo Shin. Contrastive Representation Regularization for Vision-Language-Action Models. *arXiv preprint*, 2025. 2

[21] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. Robomonkey: Scaling test-time sampling and verification for vision-language-action models. *arXiv preprint arXiv:2506.17811*, 2025. 1, 2, 3, 7, 8

[22] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024. 7

[23] Xinghang Li, Peiyan Li, Minghuan Liu, Dong Wang, Jirong Liu, Bingyi Kang, Xiao Ma, Tao Kong, Hanbo Zhang, and Huaping Liu. Towards Generalist Robot Policies: What Matters in Building Vision-Language-Action Models. *arXiv preprint arXiv:2412.14058*, 2024. 2

[24] Jijia Liu, Feng Gao, Bingwen Wei, Xinlei Chen, Qingmin Liao, Yi Wu, Chao Yu, and Yu Wang. What can rl bring to vla generalization? an empirical study. *arXiv preprint arXiv:2505.19789*, 2025. 1

[25] Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Max Du, and Chelsea Finn. Bidirectional Decoding: Improving Action Chunking via Guided Test-Time Sampling. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. 2, 5

[26] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. S1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025. 2

[27] Mitsuhiko Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. *Conference on Robot Learning (CoRL)*, 2024. 1, 2, 3

[28] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi "Jim" Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. *arXiv preprint arXiv:2503.14734*, 2025. 2

[29] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. 5

[30] Han Qi, Haocheng Yin, Aris Zhu, Yilun Du, and Heng Yang. Strengthening Generative Robot Policies through Predictive World Modeling. *arXiv preprint arXiv:2502.00622*, 2025. 3

[31] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021. 1, 4

[32] Jon Saad-Falcon, Adrian Gamarra Lafuente, Shlok Natarajan, Nahum Maru, Hristo Todorov, Etash Kumar Guha, E. Kelly Buchanan, Mayee F. Chen, Neel Guha, Christopher Re, and Azalia Mirhoseini. Archon: An Architecture Search Framework for Inference-Time Techniques. 2024. 2

[33] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[34] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. *arXiv preprint arXiv:2408.03314*, 2024. 2

[35] Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Travis Armstrong, Ashwin Balakrishna, Robert Baruch, Maria Bauza, Michiel Blokzijl, Steven Bohez, Konstantinos Bousmalis, Anthony Brohan, Thomas Buschmann, Arunkumar Byravan, Serkan Cabi, Ken Caluwaerts, Federico Casarini, Oscar Chang, Jose Enrique Chen, Xi Chen, Hao-Tien Lewis Chiang, Krzysztof Choromanski, David D'Ambrosio, Sudeep Dasari, Todor Davchev, Coline Devin, Norman Di Palo, Tianli Ding, Adil Dostmohamed, Danny Driess, Yilun Du, Debidatta Dwibedi, Michael Elabd, Claudio Fantacci, Cody Fong, Erik Frey, Chuyuan Fu, Marissa Giustina, Keerthana Gopalakrishnan, Laura Graesser, Leonard Hasenclever, Nicolas Heess, Brandon Hernaez, Alexander Herzog, R. Alex Hofer, Jan Humplik, Atil Iscen, Mithun George Jacob, Deepali Jain, Ryan Julian, Dmitry Kalashnikov, M. Emre Karagozler, Stefani Karp, Chase Kew, Jerad Kirkland, Sean Kirmani, Yuheng Kuang, Thomas Lampe, Antoine Laurens, Isabel Leal, Alex X. Lee, Tsang-Wei Edward Lee, Jacky Liang, Yixin Lin, Sharath Maddineni, Anirudha Majumdar, Assaf Hurwitz Michaely, Robert Moreno, Michael

Neunert, Francesco Nori, Carolina Parada, Emilio Parisotto, Peter Pastor, Acorn Pooley, Kanishka Rao, Krista Reymann, Dorsa Sadigh, Stefano Saliceti, Pannag Sanketi, Pierre Sermanet, Dhruv Shah, Mohit Sharma, Kathryn Shea, Charles Shu, Vikas Sindhwani, Sumeet Singh, Radu Soricut, Jost Tobias Springenberg, Rachel Sterneck, Razvan Surdulescu, Jie Tan, Jonathan Tompson, Vincent Vanhoucke, Jake Varley, Grace Vesom, Giulia Vezzani, Oriol Vinyals, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Fei Xia, Ted Xiao, Annie Xie, Jinyu Xie, Peng Xu, Sichun Xu, Ying Xu, Zhuo Xu, Yuxiang Yang, Rui Yao, Sergey Yaroshenko, Wenhao Yu, Wentao Yuan, Jingwei Zhang, Tingnan Zhang, Allan Zhou, and Yuxiang Zhou. Gemini Robotics: Bringing AI into the Physical World. *arXiv preprint arXiv:2503.20020*, 2025. 2

[36] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, et al. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025. 1, 4, 5

[37] Andrew Wagenmaker, Mitsuhiko Nakamoto, Yunchu Zhang, Seohong Park, Waleed Yagoub, Anusha Nagabandi, Abhishek Gupta, and Sergey Levine. Steering Your Diffusion Policy with Latent Space Reinforcement Learning. *arXiv preprint arXiv:2506.15799*, 2025. 2

[38] Homer Rich Walke, Kevin Black, Tony Z Zhao, Quan Vuong, Chongyi Zheng, Philippe Hansen-Estruch, Andre Wang He, Vivek Myers, Moo Jin Kim, Max Du, et al. Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning*, pages 1723–1736. PMLR, 2023. 3, 7

[39] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully Test-Time Adaptation by Entropy Minimization. In *International Conference on Learning Representations*, 2020. 2

[40] Yilin Wu, Ran Tian, Gokul Swamy, and Andrea Bajcsy. From foresight to forethought: Vlm-in-the-loop policy steering via latent alignment. In *Robotics: Science and Systems (RSS)*, 2025. 2, 3

[41] Chen Xu, Tony Khuong Nguyen, Emma Dixon, Christopher Rodriguez, Patrick Miller, Robert Lee, Paarth Shah, Rares Ambrus, Haruki Nishimura, and Masha Itkina. Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies. *arXiv preprint arXiv:2503.08558*, 2025. 1, 2, 4

[42] Shuai Yang, Hao Li, Yilun Chen, Bin Wang, Yang Tian, Tai Wang, Hanqing Wang, Feng Zhao, Yiyi Liao, and Jiangmiao Pang. InstructVLA: Vision-Language-Action Instruction Tuning from Understanding to Manipulation. *arXiv preprint*, 2025. 2

[43] Xiangcheng Zhang, Haowei Lin, Haotian Ye, James Zou, Jianzhu Ma, Yitao Liang, and Yilun Du. Inference-time Scaling of Diffusion Models through Classical Search. *arXiv preprint arXiv:2505.23614*, 2025. 2

[44] Yang Zhang, Chenwei Wang, Ouyang Lu, Yuan Zhao, Yunfei Ge, Zhenglong Sun, Xiu Li, Chi Zhang, Chenjia Bai, and Xuelong Li. Align-Then-stEer: Adapting the Vision-Language Action Models through Unified Latent Guidance. *arXiv preprint*, 2025. 2